Projekt z programowania współbieżnego

Dijkstra i delta-stepping - pełne SSSP | Pomiary: 7 listopada 2026

1. Cel i sformułowanie zadania

Celem projektu było zrównoleglenie wcześniej napisanego rozwiązania problemu najkrótszych ścieżek. Punktem odniesienia jest jednowątkowa Dijkstra z kolejką priorytetową. Wersja współbieżna wykorzystuje delta-stepping i OpenMP. Wątki współpracują przy rozwiązywaniu jednego dużego grafu; przypadki testowe nie są rozdzielane między niezależne wątki.

Dany jest graf skierowany o nieujemnych wagach całkowitych. Obliczane są odległości od wierzchołka 0 do wszystkich osiągalnych wierzchołków. Zachowano pierwotny interfejs programu: na wyjściu znajduje się odległość do n-1 albo -1. Usunięto wcześniejsze zakończenie po ustaleniu wyniku dla celu, dzięki czemu obie wersje wykonują pełne SSSP.

2. Wersja sekwencyjna i współbieżna

Dijkstra przechowuje wstępne odległości i kolejkę priorytetową. Zdejmuje wierzchołek o najmniejszej odległości, pomija nieaktualne wpisy i relaksuje jego krawędzie. Utrzymanie ścisłej kolejności odległości ogranicza możliwość bezpośredniego zrównoleglenia tego rozwiązania.

Delta-stepping [1] grupuje wierzchołki w kubełkach o przedziałach odległości $[i\Delta, (i+1)\Delta)$. Krawędzie dzieli na lekkie (waga $\leq \Delta$) i ciężkie (waga $> \Delta$). Dla najmniejszego niepustego kubełka powtarza relaksację lekkich krawędzi aż do jego opróżnienia. Następnie relaksuje ciężkie krawędzie wierzchołków zebranych podczas tej fazy. W implementacji $\Delta = \max(1, \text{floor}(2W/m))$, gdzie W jest sumą wag; dla $m=0$ przyjmuje się $\Delta=1$.

Podział pracy i synchronizacja

Pętle po wierzchołkach bieżącego frontu są dzielone przez omp for. Odległości są atomowe; compare_exchange_weak realizuje aktualizację minimum. Każdy wątek zapisuje udane relaksacje do własnego bufora. Jeden wątek, w sekcji single, scala bufory do wspólnej mapy kubełków. Domyślne bariery OpenMP oddzielają fazy pracy i scalania. Zewnętrzny region parallel utrzymuje jeden zespół wątków przez całe wywołanie.

Sekwencyjne pozostają wczytywanie, budowa grafu, podział light/heavy, inicjalizacja oraz scalanie kubełków. Koszt atomików i barier ogranicza zysk. Statyczny podział wierzchołków nie musi równoważyć liczby krawędzi przy nierównych stopniach. Front o szerokości jeden, np. ścieżka, praktycznie nie daje pracy równoległej.

3. Środowiska i metoda pomiarów

Środowisko	Laptop / WSL2	Serwer wydziałowy
Procesor	Intel Core i7-1260P	4 × Xeon Gold 6154
CPU	16 logicznych w WSL	72 rdzenie / 144 wątki sprzętowe
Użyte wątki	1, 2, 4, 8, 16	1, 2, 4, 8, 16; jeden NUMA
Kompilator	GCC 11.4, C++17, -O3	GCC 15.2, C++17, -O3
OpenMP	dynamic=FALSE, places=cores	dynamic=FALSE, places=cores

Użyto jednego grafu z 4 000 000 wierzchołków i 16 000 000 krawędzi, ziarno 20261107. Z każdego wierzchołka wychodzą cztery krawędzie: jedna tworzy skierowany cykl zapewniający osiągalność, trzy prowadzą do losowych celów. Wagi są losowane równomiernie z 1-1000. Dopuszczono pętle i krawędzie równoległe. Pliki wejściowe na obu maszynach mają identyczny SHA-256.

Dla każdej konfiguracji wykonano rozgrzewkę i pięć pomiarów, w przemieszanej kolejności. Tabele zawierają mediany. Czas compute rozpoczyna się po odczytaniu i zbudowaniu wejściowego grafu, a kończy przed wypisywaniem. Obejmuje struktury robocze, inicjalizację, obliczenia i sprzątanie; w delta-stepping także podział light/heavy oraz zebranie odległości atomowych do tablicy. Czas whole obejmuje cały proces kopii benchmarkowej.

Timery są dodawane tylko do kopii kompilowanych przez skrypt. Kopie wypisują całą tablicę odległości i każdy wynik jest porównywany z Dijkstrą poza pomiarem. Wszystkie 60 mierzonych uruchomień zakończyło się zgodnością odpowiedzi. Normalne źródła C++ zachowują wyjście dla n-1. Czas whole kopii uwzględnia pełne wyjście i nie jest pomiarem zwykłego programu z pojedynczym wynikiem.

4. Wyniki - mediany czasu obliczeń

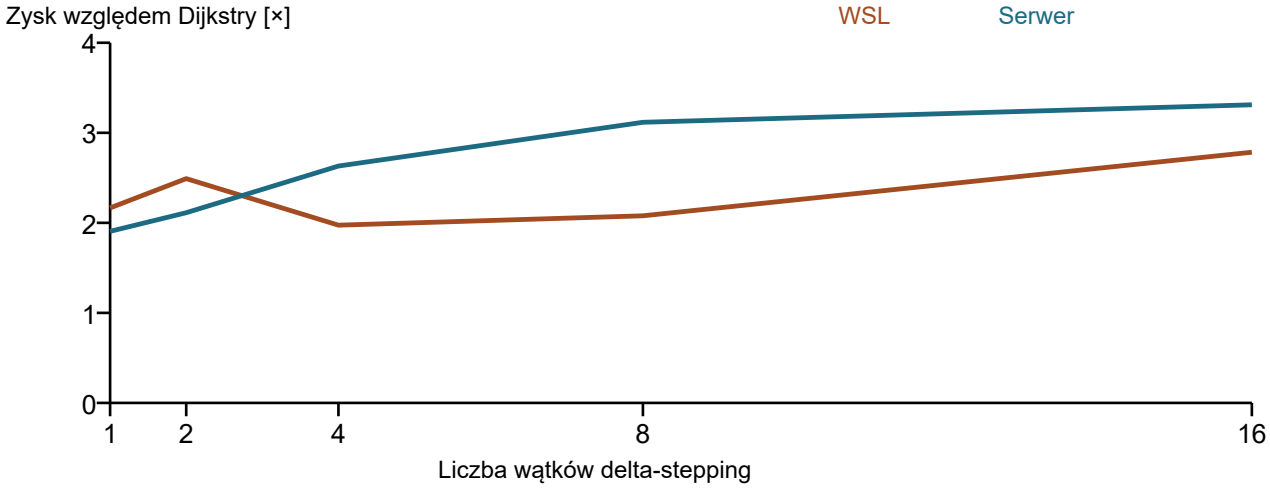
Zysk = mediana czasu Dijkstry / mediana czasu danej wersji na tej samej maszynie. W każdym polu czasu: 5 powtórzeń.

Wersja	WSL [s]	Zysk WSL	Serwer [s]	Zysk serwer
Dijkstra	8,505	1,00×	4,317	1,00×
Δ, 1 w.	3,927	2,17×	2,265	1,91×
Δ, 2 w.	3,414	2,49×	2,043	2,11×
Δ, 4 w.	4,308	1,97×	1,641	2,63×
Δ, 8 w.	4,092	2,08×	1,385	3,12×
Δ, 16 w.	3,055	2,78×	1,304	3,31×

Na serwerze czas obliczeń delta-stepping maleje wraz z liczbą wątków: z 2,265 s dla jednego do 1,304 s dla 16. Względem Dijkstry daje to 3,31×. Samo zwiększenie liczby wątków w delta-stepping daje jednak 1,74×; część przewagi pochodzi już ze zmiany algorytmu (Δ1 jest około 1,91× szybsze od Dijkstry).

Dla całego procesu kopii zysk Δ16 względem Dijkstry wyniósł 1,92× na WSL (14,913 → 7,782 s) i 1,54× na serwerze (8,276 → 5,370 s). Jest mniejszy niż dla compute. Pomiar obejmuje również etapy sekwencyjne i wypisywanie; bez osobnych timerów nie wyznaczamy dokładnego procentowego udziału każdego z nich.

5. Ocena skalowania



Na WSL najlepsza mediana obliczeń wyniosła 3,055 s przy 16 wątkach, czyli 2,78× względem Dijkstry i 1,29× względem Δ1. Skalowanie nie jest monotoniczne: 4 i 8 wątków miały mediany gorsze od Δ1. Rozrzut jest duży: Dijkstra 4,338-10,028 s, a Δ16 2,587-8,832 s. Zmienność warunków wykonania może istotnie wpływać na ilorazy median; te dane nie dowodzą trwałej przewagi 16 wątków na laptopie.

Na WSL skrypt nie wybierał jawnej listy CPU; wiązaniem wątków sterowały ustawienia OpenMP. Topologia maszyny wirtualnej nie jest pełnym opisem rdzeni fizycznych laptopa. Na serwerze użyto po jednym wątku sprzętowym z różnych rdzeni jednego procesora/NUMA. Serwer był współdzielony (load około 296-302). Różne kompilatory i polityki przypięcia ograniczają bezpośrednie porównanie sprzętu. Pięć powtórzeń jednego grafu nie zastępuje testów wielu ziaren i rodzin danych.

6. Kompilacja i uruchomienie

Linux/WSL: g++ z OpenMP. Benchmark wymaga Python 3; przypięcie CPU korzysta z taskset. Polecenia uruchamia się z katalogu projektu.

```
g++ src/dijkstraOneThread.cpp -O3 -std=c++17 -o dijkstraOneThread
g++ src/dijkstraDeltaStepping.cpp -O3 -std=c++17 -fopenmp -o dijkstraDeltaStepping
OMP_NUM_THREADS=16 OMP_DYNAMIC=FALSE ./dijkstraDeltaStepping < test.in
```

Wejście: z grafów, dla każdego n m oraz m wierszy u v w; indeksy 0..n-1 i wagi nieujemne. Benchmark generuje dane i zapisuje CSV oraz metadane w benchmark/results/.

```
python3 benchmark/scaling.py --sizes 4000000
```

7. Wnioski

Projekt zrównolegla sam algorytm dla pojedynczego przypadku. Wyniki pokazują praktyczny zysk na dużym losowym grafie, szczególnie na serwerze, oraz ograniczenia części sekwencyjnych i synchronizacji. W badanej serii uzyskano przyspieszenie 3,31× dla median czasu obliczeń względem Dijkstry. Do oceny ogólności wyniku warto użyć innych ziaren, rozmiarów oraz przypadków o wąskich frontach.

[1] U. Meyer, P. Sanders, Δ-stepping: a parallelizable shortest path algorithm, Journal of Algorithms 49(1), 114-152, 2003. https://doi.org/10.1016/S0196-6774(03)00076-2

Źródła danych: serie 20261107-222438 (WSL) i 20261107-222252 (serwer). Identyfikatory danych oraz szczegóły odtworzenia znajdują się w wersji Markdown raportu.